

ESPECIFICACIÓN TÉCNICA Y REQUISITOS FINALES DE DESARROLLO

PROYECTO: PLATAFORMA MULTI-TENANT CATALOGOHN (INQUILINO: KOLBEN)

1. DESCRIPCIÓN GENERAL DEL PROYECTO

El proyecto consiste en el desarrollo integral de **CatalogoHN**, una plataforma web orientada al modelo SaaS (*Software as a Service*) con arquitectura *multi-tenant* nativa, diseñada bajo estándares *mobile-first* para su óptima visualización y rendimiento en dispositivos móviles. El sistema permite a empresas distribuidoras gestionar y exponer sus inventarios a redes exclusivas de clientes mayoristas mediante catálogos digitales privados aislados lógicamente.

La empresa **Kolben** (importadora de cilindros hidráulicos de freno y clutch en Honduras) se constituye como el primer inquilino oficial del ecosistema. Su aplicación operará de forma independiente bajo el subdominio dinámico kolben.catalogohn.com.

El sistema automatiza el flujo de pedidos mayoristas mediante tres pilares operativos:

- **Segmentación Avanzada de Precios:** Los clientes mayoristas (~33 cuentas iniciales) acceden mediante credenciales únicas para visualizar las tarifas que les corresponden según la categoría del producto y su asignación en la matriz comercial de Kolben, manejando condiciones de crédito de 30 o 60 días.
- **Módulo Multi-Sucursal Integrado:** Permite a clientes con múltiples sedes operativas (ej. *Multi Auto* con sucursales en Comayagüela, Torocagua y San Miguel) seleccionar y añadir al carrito cantidades diferenciadas por taller en un solo flujo unificado.
- **Flujo Logístico Optimizado:** El sistema procesa el carrito del cliente, despacha notificaciones formales por correo electrónico al administrador y prepara el terreno para la validación y confirmación logística final vía WhatsApp.

2. ARQUITECTURA TÉCNICA Y STACK UTILIZADO

Se implementa una arquitectura desacoplada, segura y escalable basada en JavaScript y servicios cloud en la nube de Amazon Web Services (AWS):

Componente / Capa	Tecnología	Implementación y Alcance Técnico

Frontend	React + Vite	Aplicación SPA de alta velocidad, optimizada para consumo móvil y con estructura lista para conversión a PWA (<i>Progressive Web App</i>).
Backend	Node.js + Express	API RESTful centralizada encargada de procesar las reglas de negocio, la lógica multi-sucursal, sesiones persistentes y enrutamiento de datos.
Base de Datos	PostgreSQL (AWS RDS)	Motor relacional de nivel empresarial con aislamiento lógico de inquilinos mediante la indexación obligatoria del campo empresa_id en todas las tablas principales.
Almacenamiento	AWS S3	Repositorio de objetos seguro para almacenar las imágenes de los productos en alta resolución y los logotipos de marcas.

Hosting Cliente	AWS Amplify	Plataforma de despliegue continuo para el frontend. Provee aprovisionamiento automático, subdominios dinámicos y entornos de vista previa. Se conecta directamente al dominio ya adquirido catalogohn.com.
Hosting Servidor	AWS EC2 (t3.micro)	Instancia virtual de cómputo para la ejecución continua del backend Node.js, balanceando bajo coste inicial y escalabilidad.
Seguridad y CDN	CloudFront + ACM	Red de distribución de contenido para cargas instantáneas en telefonía móvil y cifrado estricto de tráfico mediante certificados SSL/HTTPS gratuitos aplicados sobre el dominio base y subdominios.
Notificaciones	Nodemailer + Gmail	Motor de mensajería SMTP para el envío automatizado de comprobantes de pedidos al administrador y alertas de estado al cliente.

3. CONFIGURACIÓN DE DOMINIO INICIAL Y ENTORNOS DE VISTA PREVIA

Al estar el dominio principal catalogohn.com ya comprado, la primera acción prioritaria dentro del desarrollo consistirá en el aprovisionamiento de la infraestructura cloud y la configuración del enrutamiento DNS del dominio. Esto permitirá desplegar la arquitectura multi-tenant de

forma nativa desde el primer día y habilitar un flujo de integración continua basado en **URLs de Vista Previa automáticas** aprovisionadas por AWS Amplify:

- **Enrutamiento Dinámico Inmediato:** El programador configurará las reglas de comodín (*wildcard subdomains*) sobre el dominio raíz para asegurar que cualquier subdominio creado (ej. kolben.catalogohn.com) sea resuelto y aislado lógicamente por el sistema de forma automatizada y segura.
- **Despliegues Dinámicos (Preview Deploys):** Cada actualización o ajuste que el programador realice en el código fuente generará de forma automática un enlace web de prueba temporal e independiente bajo el dominio base (ej. preview-fase1.catalogohn.com).
- **Auditoría de Diseño Interactivo:** Estos entornos web reales permitirán evaluar directamente desde smartphones el comportamiento de los layouts, fuentes tipográficas (*Barlow* y *Aquino*) y el ajuste de los filtros circulares de marcas.
- **Validación Pre-producción:** El equipo de desarrollo e inspección podrá interactuar con las nuevas funciones y remitir retroalimentación ágil antes de consolidar los cambios en el subdominio oficial, mitigando errores en el producto final.

4. ESPECIFICACIONES DE INTERFAZ Y REQUISITOS DE DISEÑO

El desarrollo técnico se ceñirá estrictamente a las directrices visuales y de interacción validadas en los documentos de diseño y archivos HTML provistos:

- **Paleta de Colores Corporativa (Kolben):** Amarillo ([#F5C200](#)) para llamadas a la acción, botones y elementos activos; Negro ([#111111](#)) para encabezados, botones secundarios y etiquetas SKU; Blanco ([#FFFFFF](#)) en tarjetas de producto ; y Gris Claro ([#F2F2F2](#)) como fondo general de la aplicación.
- **Tipografía Estricta y Bloqueo de Zoom:** Fuente *Barlow* para bloques de texto, etiquetas y botones generales ; tipografía *Aquino* para el logo y títulos principales ; e *IBM Plex Mono* para la visualización de los códigos SKU (ej. [47201-60450](#)). **Todos los campos de entrada (inputs) se fijan obligatoriamente en un tamaño base mínimo de 16px para bloquear el zoom automático nativo en dispositivos iOS (iPhone), manteniendo la app estable.**
- **Modo Oscuro Inteligente:** Activado de forma automática en el catálogo y el panel mediante la directiva CSS [@media \(prefers-color-scheme: dark\)](#). En esta modalidad, las tarjetas adoptan un fondo oscuro ([#1A1A1A](#)) y el texto vira a blanco. Las imágenes de los productos implementan un fondo claro forzado ([#F0F0F0](#) o blanco puro) con la propiedad [mix-blend-mode: multiply](#) para garantizar un contraste nítido y evitar que las fotos con fondo blanco "encandilen" al usuario en la oscuridad.
- **Filtros de Marca Circulares:** Se renderizan en formato de círculos compactos uniformes con el isotipo de la marca contenido en su interior. **Si una marca carece de logotipo en el sistema, el frontend generará dinámicamente un círculo con sus iniciales centradas en texto.** La selección activa de una marca destaca visualmente el círculo aplicando un borde amarillo.

- **Optimización y Carga de Imágenes (AWS S3):** Para proteger la cuota de almacenamiento en la nube y optimizar la velocidad en redes móviles, el backend o frontend deberá procesar y comprimir obligatoriamente cada imagen cargada por el administrador, transformándola a formatos optimizados para web (WebP o JPEG) con un peso límite estricto de **300 KB** por archivo de producto.
- **Control de Precios y Visualización en Carrito:** Al navegar por el catálogo, los totales generales no se muestran en las tarjetas para no cohibir la intención de compra. Al ingresar al carrito (panel lateral deslizable), cada ítem desglosa de forma fija e inalterable su precio unitario (**L. 450 c/u**) a la izquierda, mientras que el importe total acumulado se actualiza reactivamente a la derecha conforme se modifican las cantidades.
- **Cintas de Promoción Independientes:** Los productos marcados "En Promoción" desde el panel admin mostrarán una cinta diagonal con la palabra "PROMO" mediante CSS puro superpuesto en la esquina superior derecha, evitando la alteración o edición de los archivos de imagen originales.

5. MODELO DE ROLES Y FLUJOS FUNCIONALES DEL SISTEMA

El sistema implementará tres niveles de acceso claramente definidos con flujos de control críticos:

5.1 Roles y Permisos

- **Super Admin:** Rol global encargado de la creación, activación y suspensión de empresas inquilinas dentro del ecosistema de CatalogoHN a través del dominio principal catalogohn.com/superadmin.
- **Administrador de Empresa (Kolben - Javier):** Control total sobre el catálogo. Incluye la capacidad de crear, editar, ocultar y reordenar manualmente los productos asignando una posición numérica para controlar su orden de aparición, así como la gestión de marcas y asignación de listas de precios.
- **Cliente Mayorista:** Autenticación privada con persistencia de sesión por 30 días. Búsqueda predictiva por SKU, marca o descripción; visualización de precios personalizados; envío de pedidos y consulta de su historial de compras.

5.2 Módulo de Configuración de Marca (SaaS Preview)

En el Panel de Administración de la empresa, se implementará un **Botón / Menú de Configuración de Marca** (reemplazando la edición aislada de logo).

- **Abstracción Técnica:** Este módulo concentrará los parámetros de personalización visual de la empresa. Para el inquilino Kolben, los campos de selección de colores y fuentes tipográficas vendrán bloqueados por defecto con su diseño aprobado en el código, pero la interfaz de usuario y los endpoints del backend quedarán completamente estructurados.
- **Preparación de Base de Datos:** Los campos de formulario (Logo, Color Primario, Color Secundario y Tipo de Fuente) se mapearán directamente contra la tabla [empresas](#) en

PostgreSQL, permitiendo que en la Fase 2 la inyección de estilos CSS sea 100% dinámica en el catálogo según las variables del inquilino autenticado.

5.3 Persistencia del Carrito (Modo Resiliente a Desconexión)

Dado el enfoque mobile-first enfocado en usuarios que operan en talleres o rutas donde la señal de internet fluctúa, **el almacenamiento del carrito de compras debe guardarse localmente en el dispositivo utilizando `localStorage` o `IndexedDB`**. Si el cliente pierde la conexión momentáneamente, cierra el navegador o el dispositivo se reinicia, el estado del pedido en construcción debe persistir intacto al recuperar el acceso.

5.4 Historial de Pedidos del Cliente (Funcionalidad Incluida)

El cliente dispondrá de una sección dedicada para ver todos sus pedidos anteriores directamente desde el catálogo:

- **Detalle del Historial:** Cada registro de pedido mostrará de forma clara: número, fecha, productos, cantidades desglosadas por sucursal y estado actual.
- **Restricciones de Solo Lectura:** El cliente no puede editar ni eliminar pedidos ya enviados. Si el administrador elimina un pedido desde el panel, este desaparecerá automáticamente del historial del cliente.

5.5 Flujos Críticos de Confirmación Doble y Correos Estructurados

- **Flujo del Cliente (Envío de Pedidos):** Al presionar "Enviar Pedido", la aplicación bloquea la pantalla y despliega un modal emergente con la advertencia: *"¿Enviar pedido? Una vez enviado no podrá modificarse."*. El usuario debe optar estrictamente entre "Revisar" (retorna al carrito) o "Sí, enviar" (ejecuta el despacho).
- **Flujo del Administrator (Gestión de Estados):** Cuando el operario de Kolben modifica el estado de un pedido en su panel o ejecuta la eliminación de un registro, el sistema exige una doble confirmación afirmativa antes de impactar el backend.
- **Estructura de Notificaciones (Nodemailer):** Queda prohibido el envío de alertas en texto plano. Todos los correos despachados por Nodemailer (resumen de pedido inicial enviado a Javier, o avisos automáticos de cambio a estado **Preparando** o **Enviado** enviados al cliente) deben utilizar **plantillas HTML estructuradas y responsivas** con formato de recibo o tabla clara de productos y sucursales.

6. SEGURIDAD INFORMÁTICA Y CONTROL DE ACCESOS

Mecanismos de control aplicados para mitigar la fuga de cuentas hacia terceros o competidores del sector:

- **Cifrado de Identidades:** Procesamiento de contraseñas mediante funciones de distribución de claves con hashing seguro (`password_hash`) en PostgreSQL, erradicando el almacenamiento de datos sensibles en texto plano.
- **Mitigación de Concurrencia:** Emisión de tokens de autenticación JWT cifrados. El backend valida de forma estricta las firmas y tiempos de emisión de los tokens para

evitar la duplicación de sesiones activas en terminales no autorizadas de forma simultánea.

- **Módulo de Monitoreo de Acceso por Cliente (Auditoría):** En la sección de Clientes del Panel Admin, la ficha de cada usuario mayorista desplegará explícitamente una tarjeta de actividad en tiempo real con:
 1. Fecha y hora exacta de su último acceso exitoso al catálogo.
 2. Identificación técnica de la terminal (Dispositivo utilizado: ej. *iPhone*, *Android*, *Chrome*).
 3. Dirección IP de origen y geolocalización aproximada del punto de conexión. Esto faculta al administrador para identificar de inmediato patrones irregulares o el uso indebido de credenciales compartidas con la competencia.

7. BUENAS PRÁCTICAS DE DESARROLLO, REPOSITORIO Y SEGURIDAD (MONOREPO)

A fin de asegurar la mantenibilidad del código, aislar correctamente los entornos y garantizar que las llaves criptográficas estén protegidas, el programador debe seguir estrictamente las pautas de ingeniería de software detalladas a continuación:

7.1 Estructura del Repositorio en GitHub (Monorepo Privado)

El proyecto se manejará en un único repositorio configurado como **Privado** en GitHub. El código fuente se organizará bajo la estructura desacoplada de Monorepo:

Plaintext

```
catalogohn-platform/  # Raíz del Repositorio Privado
├── backend/           # ENTORNO DEL SERVIDOR (Node.js + Express)
│   ├── src/
│   │   ├── controllers/ # Controladores (pedidos, sucursales, auth)
│   │   ├── models/      # Esquemas y queries de PostgreSQL
│   │   ├── routes/      # Rutas y Endpoints de la API REST
│   │   └── app.js        # Punto de entrada de la aplicación
│   ├── .env            # Variables de entorno locales (OCULTO - EXCLUIDO)
│   ├── .gitignore       # Gitignore exclusivo de la capa Backend
│   └── package.json
├── frontend/          # ENTORNO DEL CLIENTE (React + Vite)
│   ├── src/
│   │   ├── components/  # Componentes UI (modales, carritos, tarjetas)
│   │   ├── pages/       # Vistas (Catálogo, Admin, SuperAdmin)
│   │   └── assets/       # Archivos estáticos (Fuentes Barlow/Aquino, Logos)
│   ├── .env            # Variables públicas del cliente (OCULTO - EXCLUIDO)
│   ├── .gitignore       # Gitignore exclusivo de la capa Frontend
│   └── package.json
└── README.md           # Documentación general de instalación y configuración
```

7.2 Gestión de Credenciales y Archivos .env

Queda estrictamente prohibido escribir contraseñas, tokens de autenticación o llaves criptográficas directamente en el código fuente (*hardcoding*).

- **Filtro Obligatorio (.gitignore):** Tanto en la carpeta **backend/** como en **frontend/**, se debe incluir un archivo **.gitignore** que contenga de forma explícita las exclusiones para evitar que las credenciales se suban al servidor Git de GitHub:
- Plaintext

```
node_modules/  
dist/  
.env  
.env.local  
*.pem
```

-
-
- **Variables del Servidor (backend/.env):** Almacenará la cadena de conexión de AWS RDS (**DATABASE_URL**), la clave de firma de tokens (**JWT_SECRET**), las llaves de acceso a AWS S3 (**AWS_ACCESS_KEY_ID** / **AWS_SECRET_ACCESS_KEY**), y los accesos SMTP de Nodemailer.
- **Variables del Cliente (frontend/.env):** Almacenará exclusivamente las URLs públicas de llamada a las APIs utilizando el prefijo requerido por el compilador (**VITE_API_URL**).
- **Entorno de Producción Cloud:** Dado que los archivos **.env** están excluidos de Git, las variables se deberán inyectar manualmente dentro del apartado *Environment Variables* de las consolas de **AWS Amplify** y **AWS EC2** al momento del aprovisionamiento.

7.3 Flujo de Trabajo en Git (Git Flow Simplificado)

Para prevenir interrupciones o caídas accidentales en la aplicación en producción que afecten la operación de Kolben, se implementa el siguiente árbol de ramificación:

1. **Rama main:** Es la rama de producción absoluta. Solo almacena código que ha sido auditado y se encuentra operando sin errores en **catalogohn.com** y sus subdominios.
2. **Rama develop:** Es la rama de integración y consolidación de avances. De esta rama se alimentan directamente los *Preview Deploys* dinámicos en AWS Amplify para las revisiones y control de calidad.
3. **Ramas feature/:** Cada nueva funcionalidad o pantalla de diseño debe programarse en una rama aislada creada desde *develop* (ej. **feature/modulo-multisucursal**). Una vez completada y probada localmente, se envía un *Pull Request* hacia la rama de desarrollo para su revisión.

7.4 Protocolo de Seguridad para Despliegue y Lanzamiento

- **Protección de Llaves de Servidor:** Las llaves privadas SSH de acceso (archivos **.pem**) utilizadas para conectarse al servidor AWS EC2 mediante consola de comandos **no** deben guardarse nunca bajo ninguna circunstancia dentro de la estructura de

carpetas del proyecto. Deben almacenarse en el directorio de llaves seguro del sistema local del programador (ej. `~/.ssh/`).

- **Autenticación Multifactor (2FA):** Obligatorio el uso de la verificación en dos pasos (2FA) en las cuentas de GitHub y consolas de AWS de todo el equipo técnico para evitar accesos no autorizados por robo de contraseñas.
- **Cifrado Global HTTPS Obligatorio:** Se debe configurar la distribución de CloudFront de manera que cualquier petición HTTP (<http://...>) entrante desde redes móviles sea redirigida de forma mandatoria con código 301 hacia la capa segura HTTPS (<https://...>), protegiendo los datos comerciales en tránsito.

8. PLANIFICACIÓN DE EXPANSIÓN A FUTURO (FASE 2)

La arquitectura de software desarrollada en esta primera etapa dejará preparadas las interfaces y abstracciones necesarias para la incorporación de los siguientes módulos comerciales a mediano plazo bajo el modelo de negocio SaaS:

- **Escalabilidad Comercial SaaS:** Apertura del dominio raíz catalogohn.com con landings de prospección y formularios de suscripción automatizada para nuevos inquilinos distribuidos de forma independiente con subdominios propios. El panel de configuración de marca se desbloqueará globalmente en la base de datos para permitir la autonomía total de colores y fuentes de terceros.
- **Abstracción para Sincronización Externa de Stock (APIs):** Disposición de capas de servicio intermedias (*gateways*) preparadas en el backend de Node.js para conectar y recibir de forma automatizada las existencias y stocks procedentes de sistemas ERP o software contable locales.
- **Procesamiento de Carga Masiva (Excel):** Módulo de parsing para la ingesta masiva de inventarios, descripciones técnicas e importación automatizada de estructuras de precios de Nivel 3 mediante archivos de Excel o CSV.
- **Herramientas Avanzadas Mayoristas:** Historial de compras optimizado con función *One-Click Reorder* (Repetir Pedido) para acelerar el abastecimiento de clientes recurrentes, junto con un sistema de alertas tempranas de desabastecimiento para SKUs con inventario por debajo del umbral de seguridad.